

Relatório sobre a atividade “18 - Prática: Visão Computacional (III)”

Lucas Gabriel Arenhardt

1. Introdução

Neste módulo foram abordados vários tipos de redes neurais - desde as já vistas anteriormente no bootcamp (redes neurais simples, sequenciais e convolucionais), até algumas novas (Auto Encoders e Variational Auto Encoders) - e suas aplicações utilizando o PyTorch. Como muitos conceitos desse módulo já foram explicados em relatórios anteriores, abaixo estão apresentados somente os que ainda não foram trabalhados durante o Bootcamp.

2. Conceitos

2.1. PyTorch

PyTorch é uma biblioteca de código aberto para aprendizado de máquina e computação numérica, amplamente usada para construir e treinar redes neurais. Originalmente desenvolvida pela Meta AI, ela se destaca pelo uso de tensores com suporte a GPU. O PyTorch é uma biblioteca robusta e, assim como o TensorFlow, é uma das mais usadas para Machine Learning.

2.2. Tensor

Um tensor no PyTorch é uma estrutura de dados fundamental que representa um array multidimensional, semelhante a arrays do NumPy, mas com suporte adicional para operações em GPUs. Tensores podem ter diversas dimensões, desde um escalar (0D), vetor (1D), matriz (2D), até tensores de dimensões mais altas. Para converter dados para tensor, basta utilizar a função `torch.tensor()`

É possível realizar algumas operações com os tensores PyTorch, como multiplicação, adição, permutação e redimensionamento.

A função `torch.squeeze()` remove as dimensões de tamanho 1 em um tensor. Já a `torch.unsqueeze()` adiciona dimensões unitárias ao mesmo.

Tensores também podem ser criados com as funções `torch.zeros()`, `torch.ones()` e `torch.randn()`.

2.3. Construção da Rede Neural no PyTorch

A construção da rede neural no PyTorch é um pouco menos intuitiva do que no TensorFlow, porém os fundamentos são os mesmos.

Primeiramente é necessário criar uma classe que herde de “nn.module”. No método “__init__” são definidas as camadas da rede e no método “forward” é descrito como os dados passam pelas camadas.

```
#criando a rede neural

class MyNeuralNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(2,8)
        self.activation = nn.ReLU()
        self.layer2 = nn.Linear(8,1)

    def forward(self,x):
        x = self.layer1(x)
        x = self.activation(x)
        x = self.layer2(x)
        return x
```

Imagem 1 - Definindo uma rede neural com o PyTorch.

Em seguida, cria-se um objeto da classe definida. Deve-se também escolher a função de perda e o otimizador. Por fim, no loop de treinamento, deve-se passar os dados de entrada pelo modelo, calcular a perda, realizar a retropropagação com “loss.backward()” e atualizar os pesos com optimizer.step(). Esses são os passos para a construção de uma rede neural básica no PyTorch.

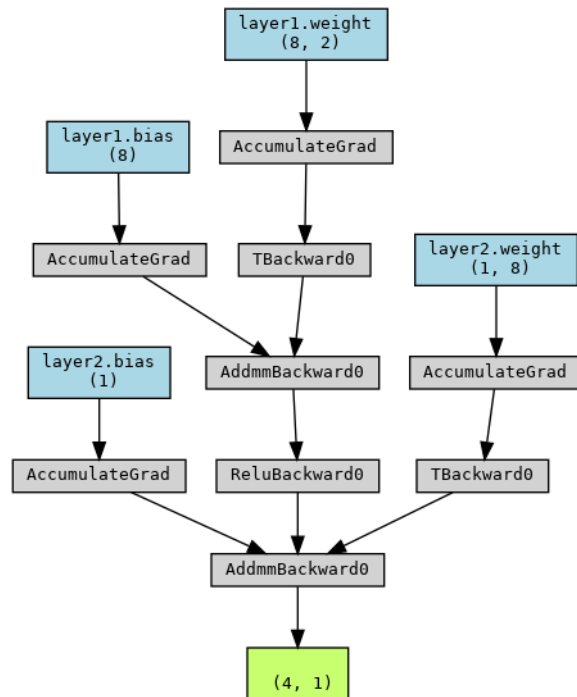


Imagem 2 - Estrutura de uma rede neural.

2.4. OpenCV

Em uma das sessões do curso, foi mostrada de maneira breve a biblioteca OpenCV. O OpenCV (Open Source Computer Vision Library) é uma biblioteca de software de código aberto voltada para a computação visual e aprendizado de máquina. Desenvolvida originalmente pela Intel, e agora mantida por uma comunidade global, o OpenCV oferece uma ampla gama de funcionalidades para a captura, processamento e análise de imagens e vídeos.

2.5. Autoencoder

Autoencoder é um tipo de rede neural utilizada em tarefas de aprendizado não supervisionado, como compressão de dados e redução de dimensionalidade. Uma rede neural do tipo autoencoder deve ser capaz de aprender uma representação compacta dos dados de entrada e também reconstruir as informações originais a partir dessa representação.

No caso das imagens, os autoencoders recebem uma imagem de entrada e aplicam uma série de operações, como convoluções, pooling e ativações. A saída do encoder é o espaço latente, uma representação compacta da imagem original. Em seguida, o decoder realiza as operações inversas às do encoder, para tentar reconstruir a imagem.

```

class ConvAutoEncoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 32, 3, stride=3, padding=1), nn.ReLU(True),
            nn.MaxPool2d(2, stride=2),
            nn.Conv2d(32, 64, 3, stride=2, padding=1), nn.ReLU(True),
            nn.MaxPool2d(2, stride=1)
        )
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(64, 32, 3, stride=2), nn.ReLU(True),
            nn.ConvTranspose2d(32, 16, 5, stride=3, padding=1), nn.ReLU(True),
            nn.ConvTranspose2d(16, 1, 2, stride=2, padding=1), nn.Tanh()
        )
    def forward(self, x):
        x = self.encoder(x)
        x = self.decoder(x)
        return x

```

Imagem 3 - Construção de um Autoencoder.

2.5.1. Variational Autoencoders (VAEs)

Os VAEs são uma extensão dos Autoencoders tradicionais. Além de realizarem a compressão e descompressão dos dados, eles também são capazes de gerar novos dados similares aos dados de entrada.

A principal diferença entre autoencoders tradicionais e variational autoencoders (VAEs) está na maneira como eles lidam com o espaço latente. Nos autoencoders tradicionais, a entrada é comprimida em um único ponto no espaço latente. Já os VAEs tratam esse espaço como uma distribuição probabilística, representada por dois vetores: um de médias e outro de desvios padrão. Isso permite que os VAEs possam amostrar diferentes pontos desse espaço para gerar novas entradas que se assemelham às originais, tornando-os muito úteis para a geração de imagens, por exemplo.

3. Conclusão

Este módulo permitiu a exploração da construção e uso de diferentes tipos de redes neurais utilizando o PyTorch. O conhecimento adquirido com o PyTorch, uma das principais bibliotecas de aprendizado de máquina, complementou as habilidades já desenvolvidas com o TensorFlow, proporcionando uma nova perspectiva na criação e no treinamento de redes neurais.

O estudo dos Autoencoders e VAEs permitiu compreender como essas redes podem, além de reduzir a dimensionalidade e realizar compressão de dados, gerar novas amostras a partir de distribuições probabilísticas no espaço latente. Mesmo que breve, a apresentação sobre o OpenCV abre possibilidades para a integração das redes neurais com técnicas avançadas de processamento de

imagem. Em conjunto, esses conhecimentos ampliam significativamente as capacidades para desenvolver modelos mais eficientes e inovadores em aprendizado de máquina e visão computacional.